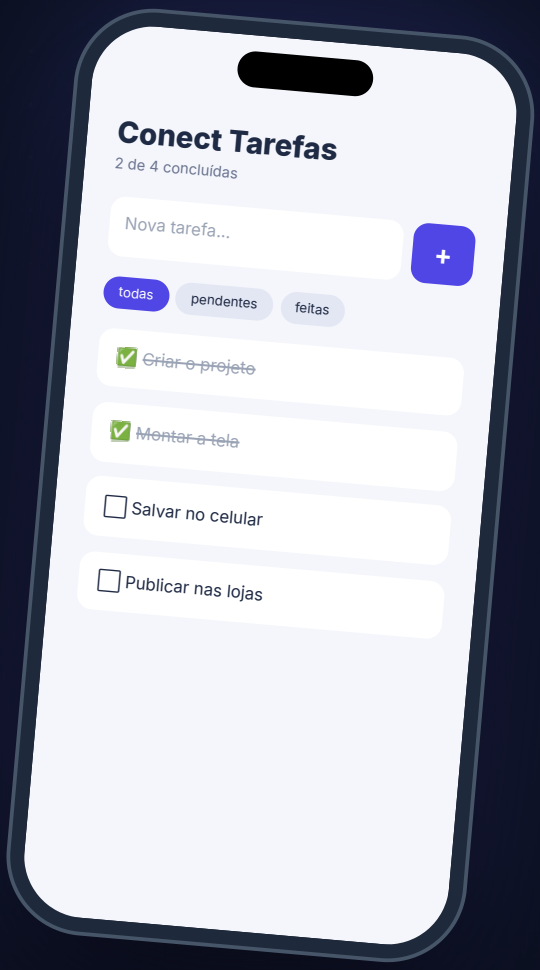
WORKSHOP CONECT INOVE

Do zero à loja

Seu primeiro app com React Native e Expo, da primeira linha de código às lojas de aplicativos

Encontro 1	Encontro 2	Encontro 3	Encontro 4	Encontro 5
26/10	27/10	29/10	05/11	07/11
Seg 19:30	Ter 19:30	Qui 19:30	Qui 19:30	Sáb 13:00



Apostila do aluno

Professor: Otávio Rafael

5 encontros de 1h pelo Google Meet

github.com/otaviorafaeldev/workshop-conect-inove-react-native

0

COMECE POR AQUI

Mapa e preparação

ANTES DO DIA 26/10

NESTE CAPÍTULO

1 Mapa do workshop

2 Antes do encontro 1: checklist de instalação

Mapa do workshop

ENCONTRO	DATA	HORÁRIO	O QUE VAMOS FAZER
1	Seg, 26/10	19:30 às 20:30	Preparar o ambiente, criar o projeto e montar a tela do app
2	Ter, 27/10	19:30 às 20:30	Deixar o app vivo: estado, adicionar, concluir e apagar tarefas
3	Qui, 29/10	19:30 às 20:30	Salvar no celular, filtros, nome, ícone e configuração para as lojas
4	Qui, 05/11	19:30 às 20:30	Gerar o app (build) e publicar na Google Play
5	Sáb, 07/11	13:00 às 14:00	Como publicar na Apple App Store (demonstrativo), atualizações e próximos passos

O que você vai aprender

- O que são React Native e Expo e por que usar os dois juntos
- Componentes, estilos, estado (`useState`) e efeitos (`useEffect`)
- Listas com `FlatList` e armazenamento local com `AsyncStorage`
- O caminho completo de um app até as lojas: contas de desenvolvedor, build, assinatura, testes, ficha da loja, política de privacidade e revisão

Antes do encontro 1: checklist de instalação

Faça isto **antes** de 26/10 para não gastar a aula com instalação.

ITEM	PARA QUE SERVE	ONDE PEGAR
Node.js (versão LTS)	Roda o JavaScript fora do navegador e traz o `npm`	nodejs.org
VS Code	Editor de código	code.visualstudio.com
Git	Controle de versões: guarda o histórico do seu código	git-scm.com (teste com `git --version`)
Conta gratuita no GitHub	Baixar o projeto e guardar seus desafios online	github.com
Celular com o app Expo Go	Ver seu app rodando na hora	Play Store ou App Store, busque "Expo Go"
Conta gratuita no Expo	Usada nos builds (encontro 4 em diante)	expo.dev/signup
Conta Google (Gmail)	Google Meet e, depois, Play Console	você já tem


Node.js


VS Code


Git


Conta GitHub


Conta Expo

Teste se deu certo. Abra o terminal (no Windows, o "Prompt de Comando" ou "PowerShell"; no Mac/Linux, o "Terminal") e digite:

```
terminal

node -v
npm -v
```

Se aparecerem números de versão (ex.: `v24.16.0`), está pronto. Se aparecer "comando não encontrado", feche e abra o terminal de novo ou reinstale o Node.

Celular e computador devem estar no mesmo Wi-Fi durante as aulas de código.

ENCONTRO 1

Ambiente e primeira tela

SEG 26/10 · 19:30 ÀS 20:30

NESTE ENCONTRO

1. O que é cada coisa?
2. O caminho completo do app
3. Criando o projeto
4. Conhecendo os arquivos
5. Os blocos de construção
6. Montando a tela (Etapa 1)
7. Git e GitHub: pegando o projeto e guardando seus desafios

1.1 O que é cada coisa?

- **React Native**: biblioteca criada pelo Facebook (Meta) para fazer apps de celular usando JavaScript e React. Você escreve uma vez e roda no Android e no iPhone.
- **Expo**: um conjunto de ferramentas em cima do React Native. Ele cuida da parte chata (configurar Android Studio, Xcode, certificados) e deixa você testar o app em segundos.
- **Expo Go**: app que você instala no celular para abrir seu projeto durante o desenvolvimento.
- **EAS (Expo Application Services)**: serviço na nuvem do Expo que **gera o arquivo final do app** e ajuda a enviar para as lojas. Vamos usar nos encontros 4 e 5.

1.2 O caminho completo do app

```texto
Código (VS Code) → Testar (Expo Go) → Build (EAS) → Loja (Play / App Store) → Revisão → Usuários
```

Guarde esse mapa. Nos encontros 1 a 3 percorremos as duas primeiras etapas. Nos encontros 4 e 5 percorremos o resto.

1.3 Criando o projeto

No terminal, vá até a pasta onde você guarda seus projetos e rode:

```terminal
npx create-expo-app@latest conect-tarefas --template blank
cd conect-tarefas
npx expo start
```

O que cada comando faz:

COMANDO	O QUE FAZ
`npx create-expo-app@latest`	Baixa e roda o gerador de projetos do Expo, sempre na versão mais nova
`conect-tarefas`	Nome da pasta do projeto
`--template blank`	Projeto mínimo, só com o essencial
`cd conect-tarefas`	Entra na pasta criada
`npx expo start`	Liga o servidor de desenvolvimento e mostra um **QR Code**

Abrindo no celular

- **Android:** abra o Expo Go e toque em "Scan QR code".
- **iPhone:** abra a **Câmera** do celular, aponte para o QR Code e toque na notificação.

Apareceu o texto "Open up App.js to start working on your app!"? Seu primeiro app está rodando.

Se o QR Code não funcionar

1. Confirme que celular e computador estão no mesmo Wi-Fi.
2. Pare o servidor (`Ctrl + C`) e rode `npx expo start --tunnel` . Se pedir para instalar algo, digite `y` .
3. Limpe o cache: `npx expo start --clear` .
4. Plano B: no terminal do servidor, aperte `w` para abrir no navegador.

1.4 Conhecendo os arquivos

Abra a pasta no VS Code (`File > Open Folder`). Os arquivos importantes:

ARQUIVO	FUNÇÃO
`App.js`	A tela principal do app. É aqui que vamos trabalhar
`app.json`	Configurações do app: nome, ícone, identificador, versão
`package.json`	Lista as bibliotecas (dependências) do projeto
`assets/`	Imagens: ícone, splash, etc.
`node_modules/`	Bibliotecas baixadas. Nunca mexa e nunca envie para ninguém

1.5 Os blocos de construção

No React Native não existe `<div>` nem `<p>` . Usamos **componentes** próprios:

COMPONENTE	É COMO...	PARA QUE SERVE
`View`	uma caixa (`div`)	Agrupar e organizar coisas
`Text`	um parágrafo	**Todo** texto precisa estar dentro de um `Text`
`TextInput`	um campo de digitação	Receber texto do usuário
`TouchableOpacity`	um botão	Qualquer coisa que reage ao toque
`FlatList`	uma lista	Mostrar muitos itens com rolagem (encontro 2)
`StyleSheet`	um CSS	Definir cores, tamanhos, espaçamentos

Regras de estilo importantes:

- Os nomes são em camelCase: `backgroundColor` , `fontSize` , `marginBottom` .
- Valores numéricos não levam "px": `fontSize: 16` .
- O layout usa **Flexbox**, e o padrão é em **coluna** (de cima para baixo). Para ficar lado a lado: `flexDirection: 'row'` .

1.6 Montando a tela (Etapa 1)

Apague tudo o que está em `App.js` e cole:

App.js

```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, TextInput, TouchableOpacity } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.titulo}>Conect Tarefas</Text>

      <View style={styles.linhaNova}>
        <TextInput style={styles.input} placeholder="Nova tarefa..." />
        <TouchableOpacity style={styles.botao}>
          <Text style={styles.botaoTexto}>+</Text>
        </TouchableOpacity>
      </View>

      <View style={styles.card}>
        <Text style={styles.cardTexto}>Estudar React Native</Text>
      </View>
      <View style={styles.card}>
        <Text style={styles.cardTexto}>Publicar meu app</Text>
      </View>

      <StatusBar style="auto" />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#F4F6FB', paddingTop: 70, paddingHorizontal: 20 },
  titulo: { fontSize: 30, fontWeight: 'bold', color: '#1F2A44', marginBottom: 20 },
  linhaNova: { flexDirection: 'row', marginBottom: 20 },
  input: { flex: 1, backgroundColor: '#fff', borderRadius: 10, paddingHorizontal: 14, fontSize: 16
},
  botao: { backgroundColor: '#4F46E5', width: 50, borderRadius: 10, marginLeft: 10, alignItems:
'center', justifyContent: 'center' },
  botaoTexto: { color: '#fff', fontSize: 28 },
  card: { backgroundColor: '#fff', padding: 16, borderRadius: 10, marginBottom: 10 },
  cardTexto: { fontSize: 16, color: '#1F2A44' },
});
```

Salve (`Ctrl + S`). O app no celular atualiza sozinho (isso se chama **Fast Refresh**).

Entendendo o código:

- `export default function App()` é o **componente** principal. Um componente é uma função que devolve a tela.
- O que está dentro do `return (...)` se chama **JSX**: uma mistura de JavaScript com marcação parecida com HTML.
- `style={styles.titulo}` aplica um estilo definido lá embaixo no `StyleSheet.create`.
- `flex: 1` significa "ocupe todo o espaço disponível".

🏆 **Desafio 1:** troque as cores do app (o roxo `#4F46E5` e o fundo `#F4F6FB`), mude o título para o seu nome e adicione um terceiro card. Mande um print no grupo e salve o resultado em um commit na sua branch (veja a seção 1.7).

1.7 Git e GitHub: pegando o projeto e guardando seus desafios

O que é Git? E o GitHub?

- **Git** é um programa que guarda o **histórico** do seu código. Pense em um "salvar jogo": cada *commit* é um ponto de salvamento para onde você pode voltar.
- **GitHub** é um site que guarda repositórios (projetos com Git) na internet, para compartilhar e fazer backup.
- **Repositório** é a pasta do projeto com todo o histórico. **Branch** (ramo) é uma linha paralela de trabalho: você mexe na sua branch sem estragar a original (`main`).

Repositório do workshop

https://github.com/otaviorafaeldev/workshop-conect-inove-react-native

Repositório do workshop

Lá estão o projeto completo (`conect-tarefas/`), as etapas de cada aula (`conect-tarefas/etapas/`) e esta apostila em PDF. Você pode usar o projeto como **ponto de resgate** (se travar na aula) e como **base para os desafios**.

Configure o Git uma vez (primeira vez no computador)

terminal

```
git config --global user.name "Seu Nome"
git config --global user.email "seuemail@exemplo.com"
```

Use o mesmo e-mail da sua conta do GitHub.

Opção 1: baixar o ZIP (mais simples, sem Git)

1. Abra o link do repositório no navegador.
2. Clique no botão verde **Code** e depois em **Download ZIP**. Ou baixe direto: https://github.com/otaviorafaeldev/workshop-conect-inove-react-native/archive/refs/heads/main.zip
3. Extraia o ZIP, abra a pasta `conect-tarefas` no terminal e rode:

```
terminal
npm install
npx expo start
```

Dica: o `npm install` baixa as bibliotecas (a pasta `node_modules`). Ela não vem no ZIP e não precisa vir.

Opção 2: clonar com Git (recomendado)

```
terminal
git clone https://github.com/otaviorafaeldev/workshop-conect-inove-react-native.git
cd workshop-conect-inove-react-native/conect-tarefas
npm install
npx expo start
```

Criando a sua branch para os desafios

Nunca resolva os desafios direto na `main`. Crie uma branch com o **seu nome** (minúsculas, sem acentos e sem espaços, separando com hífen):

```
terminal
git switch -c desafios/maria-silva
git branch --show-current
```

- `git switch -c` cria a branch e já entra nela.
- `git branch --show-current` mostra em qual branch você está. Deve aparecer `desafios/maria-silva`.

Salvando o desafio (commit)

Depois de resolver o desafio e testar no celular:

```
terminal
git status
git add .
git commit -m "Desafio 1: troquei as cores e o titulo"
git log --oneline
```

COMANDO	O QUE FAZ
`git status`	Mostra o que você alterou
`git add .`	Separa todas as alterações para o próximo salvamento
`git commit -m "mensagem"`	Cria o ponto de salvamento, com uma mensagem curta dizendo o que fez
`git log --oneline`	Lista os salvamentos feitos

Faça **um commit por desafio** (`Desafio 2: ...`, `Desafio 3: ...`), sempre na sua branch.

Alternando entre a versão original e a sua

```terminal
git switch main                    # versao original do professor
git switch desafios/maria-silva    # a sua versao
```

Antes de trocar de branch, faça o commit (ou o Git avisa que há alterações não salvas).

Guardando online no seu GitHub (opcional, mas recomendado)

O repositório do professor é somente leitura para você: se você tentar `git push` nele, vai aparecer um erro de permissão (isso é normal). Para guardar a sua branch online, use um **fork** (uma cópia do repositório na sua conta):

1. Abra o repositório do workshop no GitHub e clique em **Fork** (canto superior direito) e depois em **Create fork**.
2. No terminal, dentro da pasta do projeto, ligue o seu fork e envie a sua branch:

```terminal
git remote add meu-fork https://github.com/SEU-USUARIO/workshop-conect-inove-react-native.git
git push -u meu-fork desafios/maria-silva
```

Troque `SEU-USUARIO` pelo seu usuário do GitHub e `maria-silva` pelo seu nome. Se pedir login: o GitHub não aceita mais senha comum. Use `gh auth login` (GitHub CLI), o app GitHub Desktop, ou um token de acesso pessoal.

Se você já clonou direto do seu fork, o remoto se chama `origin` e o comando é `git push -u origin desafios/maria-silva`.

Se o professor atualizar o repositório

```terminal
git switch main
git pull
git switch desafios/maria-silva
git merge main
```

Se aparecer conflito, chame o professor ou peça ajuda no grupo.

Cola rápida de Git

QUERO...	COMANDO
Ver em qual branch estou	`git branch --show-current`
Ver o que mudei	`git status`
Salvar meu trabalho	`git add .` e `git commit -m "mensagem"`
Ver o histórico	`git log --oneline`

QUERO...	COMANDO
Descartar alterações **não salvas** de um arquivo	`git restore App.js`
Voltar para a versão do professor	`git switch main`

⚠️ **Atenção:** `git restore` apaga as alterações que ainda não foram salvas em commit. Use com cuidado.

ENCONTRO 2

Deixando o app vivo

TER 27/10 · 19:30 ÀS 20:30

NESTE ENCONTRO

1. Revisão rápida
2. O que é estado (`useState`)
3. Etapa 2: adicionar, concluir e apagar
4. Entendendo cada peça

2.1 Revisão rápida

Rode de novo `npx expo start` dentro da pasta do projeto e abra no celular. Tudo certo? Então vamos dar vida ao app.

2.2 O que é estado (`useState`)

Estado é a memória de um componente. Quando o estado muda, a tela se redesenha sozinha.

App.js

```
const [texto, setTexto] = useState('');
```

- `texto` é o valor atual.
- `setTexto` é a função que **muda** o valor (nunca altere `texto` diretamente).
- `''` é o valor inicial (um texto vazio).

Para uma lista de tarefas, o estado é uma lista (array) de objetos:

App.js

```
const [tarefas, setTarefas] = useState([]);
// cada tarefa: { id: '123', titulo: 'Estudar', feita: false }
```

2.3 Etapa 2: adicionar, concluir e apagar

Substitua o `App.js` por:

App.js

```
import { useState } from 'react';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, TextInput, TouchableOpacity, FlatList } from 'react-native';

export default function App() {
  const [texto, setTexto] = useState('');
  const [tarefas, setTarefas] = useState([]);

  function adicionar() {
    if (texto.trim() === '') return;
    const nova = { id: Date.now().toString(), titulo: texto.trim(), feita: false };
    setTarefas([nova, ...tarefas]);
    setTexto('');
  }

  function alternar(id) {
    setTarefas(tarefas.map((t) => (t.id === id ? { ...t, feita: !t.feita } : t)));
  }

  function apagar(id) {
    setTarefas(tarefas.filter((t) => t.id !== id));
  }
```

```
  return (
    <View style={styles.container}>
      <Text style={styles.titulo}>Conect Tarefas</Text>

      <View style={styles.linhaNova}>
        <TextInput
          style={styles.input}
          placeholder="Nova tarefa..."
          value={texto}
          onChangeText={setTexto}
          onSubmitEditing={adicionar}
        />
        <TouchableOpacity style={styles.botao} onPress={adicionar}>
          <Text style={styles.botaoTexto}>+</Text>
        </TouchableOpacity>
      </View>

      <FlatList
        data={tarefas}
        keyExtractor={(item) => item.id}
        ListEmptyComponent={<Text style={styles.vazio}>Nenhuma tarefa ainda</Text>}
        renderItem={({ item }) => (
          <View style={styles.card}>
            <TouchableOpacity style={{ flex: 1 }} onPress={() => alternar(item.id)}>
              <Text style={[styles.cardTexto, item.feita && styles.feita]}>
                {item.feita ? '✅ ' : '⬜ '}
                {item.titulo}
              </Text>
            </TouchableOpacity>
            <TouchableOpacity onPress={() => apagar(item.id)}>
              <Text style={styles.lixo}>🗑️</Text>
            </TouchableOpacity>
          </View>
        )}
      />

      <StatusBar style="auto" />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#F4F6FB', paddingTop: 70, paddingHorizontal: 20 },
  titulo: { fontSize: 30, fontWeight: 'bold', color: '#1F2A44', marginBottom: 20 },
  linhaNova: { flexDirection: 'row', marginBottom: 20 },
  input: { flex: 1, backgroundColor: '#fff', borderRadius: 10, paddingHorizontal: 14, fontSize: 16
},
  botao: { backgroundColor: '#4F46E5', width: 50, borderRadius: 10, marginLeft: 10, alignItems:
'center', justifyContent: 'center' },
  botaoTexto: { color: '#fff', fontSize: 28 },
  card: { backgroundColor: '#fff', padding: 16, borderRadius: 10, marginBottom: 10, flexDirection:
'row', alignItems: 'center' },
  cardTexto: { fontSize: 16, color: '#1F2A44' },
  feita: { textDecorationLine: 'line-through', color: '#9AA3B5' },
  lixo: { fontSize: 20, marginLeft: 10 },
```

```
  vazio: { textAlign: 'center', color: '#9AA3B5', marginTop: 40 },
});
```

2.4 Entendendo cada peça

`adicionar()`

1. `texto.trim() === ''` ignora tarefas vazias.
2. Cria um objeto com `id` único (a hora atual em milissegundos), o título e `feita: false`.
3. `setTarefas([nova, ...tarefas])` cria uma lista nova com a tarefa no começo. O `...tarefas` "espalha" os itens antigos.
4. `setTexto('')` limpa o campo.

`alternar(id)` usa `map` para percorrer a lista. Na tarefa com aquele `id`, troca `feita` pelo contrário (`!t.feita`); as outras ficam iguais.

`apagar(id)` usa `filter` para manter só as tarefas com `id` diferente do apagado.

⚠️ **Regra de ouro do React:** nunca mexa direto na lista (`tarefas.push(...)` está errado). Sempre crie uma lista nova e entregue para o `setTarefas`.

`FlatList` recebe três coisas principais:

- `data`: a lista de dados;
- `keyExtractor`: como identificar cada item de forma única;
- `renderItem`: o que desenhar para cada item.

Evento `onPress` é o "ao tocar". Usamos `() => alternar(item.id)` para passar qual item foi tocado.

Estilo condicional: `[styles.cardTexto, item.feita && styles.feita]` aplica o risco no texto só quando a tarefa está feita.

🏆 **Desafio 2 (lembre do commit na sua branch):** adicione um texto que mostre quantas tarefas existem (`{tarefas.length}`) embaixo do título. Bônus: mude a cor do card quando a tarefa estiver feita.

3

ENCONTRO 3

Salvar no celular e preparar para as lojas

QUI 29/10 · 19:30 ÀS 20:30

NESTE ENCONTRO

1. O problema
2. Instalando o AsyncStorage
3. Efeitos (`useEffect`)
4. Etapa 3: versão final do App.js
5. Preparando a identidade do app (`app.json`)

3.1 O problema

Feche o app e abra de novo: as tarefas sumiram. O estado vive só na memória. Precisamos **guardar no celular**.

3.2 Instalando o AsyncStorage

Pare o servidor (`Ctrl + C`) e instale:

terminal

```
npx expo install @react-native-async-storage/async-storage
```

Use `npx expo install` (e não `npm install`) para bibliotecas do Expo: ele escolhe a versão compatível com o seu projeto. Depois rode `npx expo start` novamente.

3.3 Efeitos (`useEffect`)

`useEffect` executa um código em momentos específicos:

App.js

```
useEffect(() => {
  // código
}, [dependencias]);
```

- Com a lista vazia `[]` : roda **uma vez**, quando o app abre.
- Com `[tarefas]` : roda **toda vez** que `tarefas` mudar.

Usamos dois efeitos: um para **ler** ao abrir o app e outro para **salvar** quando a lista muda. O AsyncStorage guarda apenas texto, então usamos `JSON.stringify` (objeto para texto) ao salvar e `JSON.parse` (texto para objeto) ao ler.

3.4 Etapa 3: versão final do App.js

App.js

```
import { useState, useEffect } from 'react';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, TextInput, TouchableOpacity, FlatList } from 'react-native';
import AsyncStorage from '@react-native-async-storage/async-storage';

const CHAVE = '@conect_tarefas';

export default function App() {
  const [texto, setTexto] = useState('');
  const [tarefas, setTarefas] = useState([]);
  const [carregou, setCarregou] = useState(false);
  const [filtro, setFiltro] = useState('todas'); // 'todas' | 'pendentes' | 'feitas'

  // Ao abrir o app: le as tarefas salvas
```

```
  useEffect(() => {
    async function carregar() {
      try {
        const salvo = await AsyncStorage.getItem(CHAVE);
        if (salvo) setTarefas(JSON.parse(salvo));
      } catch (e) {
        console.log('Erro ao carregar', e);
      }
      setCarregou(true);
    }
    carregar();
  }, []);

  // Sempre que a lista mudar: salva (so depois de carregar a primeira vez)
  useEffect(() => {
    if (carregou) AsyncStorage.setItem(CHAVE, JSON.stringify(tarefas));
  }, [tarefas, carregou]);

  function adicionar() {
    if (texto.trim() === '') return;
    const nova = { id: Date.now().toString(), titulo: texto.trim(), feita: false };
    setTarefas([nova, ...tarefas]);
    setTexto('');
  }

  function alternar(id) {
    setTarefas(tarefas.map((t) => (t.id === id ? { ...t, feita: !t.feita } : t)));
  }

  function apagar(id) {
    setTarefas(tarefas.filter((t) => t.id !== id));
  }

  const visiveis = tarefas.filter((t) => {
    if (filtro === 'pendentes') return !t.feita;
    if (filtro === 'feitas') return t.feita;
    return true;
  });
  const qtdFeitas = tarefas.filter((t) => t.feita).length;

  return (
    <View style={styles.container}>
      <Text style={styles.titulo}>Conect Tarefas</Text>
      <Text style={styles.contador}>
        {qtdFeitas} de {tarefas.length} concluídas
      </Text>

      <View style={styles.linhaNova}>
        <TextInput
          style={styles.input}
          placeholder="Nova tarefa..."
          value={texto}
          onChangeText={setTexto}
          onSubmitEditing={adicionar}
        />
        <TouchableOpacity style={styles.botao} onPress={adicionar}>
          <Text style={styles.botaoTexto}>+</Text>
```

```
        </TouchableOpacity>
      </View>

      <View style={styles.filtros}>
        {['todas', 'pendentes', 'feitas'].map((f) => (
          <TouchableOpacity
            key={f}
            style={[styles.filtro, filtro === f && styles.filtroAtivo]}
            onPress={() => setFiltro(f)}
          >
            <Text style={[styles.filtroTexto, filtro === f && styles.filtroTextoAtivo]}>{f}</Text>
          </TouchableOpacity>
        ))}
      </View>

      <FlatList
        data={visiveis}
        keyExtractor={(item) => item.id}
        ListEmptyComponent={<Text style={styles.vazio}>Nenhuma tarefa aqui</Text>}
        renderItem={({ item }) => (
          <View style={styles.card}>
            <TouchableOpacity style={{ flex: 1 }} onPress={() => alternar(item.id)}>
              <Text style={[styles.cardTexto, item.feita && styles.feita]}>
                {item.feita ? '✅ ' : '⬜ '}
                {item.titulo}
              </Text>
            </TouchableOpacity>
            <TouchableOpacity onPress={() => apagar(item.id)}>
              <Text style={styles.lixo}>🗑️</Text>
            </TouchableOpacity>
          </View>
        )}
      />

      <StatusBar style="auto" />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#F4F6FB', paddingTop: 70, paddingHorizontal: 20 },
  titulo: { fontSize: 30, fontWeight: 'bold', color: '#1F2A44' },
  contador: { color: '#6B7490', marginBottom: 20, marginTop: 4 },
  linhaNova: { flexDirection: 'row', marginBottom: 16 },
  input: { flex: 1, backgroundColor: '#fff', borderRadius: 10, paddingHorizontal: 14, fontSize: 16
},
  botao: { backgroundColor: '#4F46E5', width: 50, borderRadius: 10, marginLeft: 10, alignItems:
'center', justifyContent: 'center' },
  botaoTexto: { color: '#fff', fontSize: 28 },
  filtros: { flexDirection: 'row', marginBottom: 16 },
  filtro: { paddingVertical: 6, paddingHorizontal: 14, borderRadius: 20, backgroundColor: '#E3E7F3',
marginRight: 8 },
  filtroAtivo: { backgroundColor: '#4F46E5' },
  filtroTexto: { color: '#1F2A44', textTransform: 'capitalize' },
  filtroTextoAtivo: { color: '#fff' },
  card: { backgroundColor: '#fff', padding: 16, borderRadius: 10, marginBottom: 10, flexDirection:
'row', alignItems: 'center' },
```

```
  cardTexto: { fontSize: 16, color: '#1F2A44' },
  feita: { textDecorationLine: 'line-through', color: '#9AA3B5' },
  lixo: { fontSize: 20, marginLeft: 10 },
  vazio: { textAlign: 'center', color: '#9AA3B5', marginTop: 40 },
});
```

Teste: adicione tarefas, feche o Expo Go completamente, abra de novo. As tarefas continuam lá? O app está pronto.

3.5 Preparando a identidade do app (`app.json`)

Antes de publicar, o app precisa ter nome, ícone e **identificadores únicos**. Edite o `app.json`:

app.json

```
{
  "expo": {
    "name": "Conect Tarefas",
    "slug": "conect-tarefas",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "userInterfaceStyle": "light",
    "ios": {
      "supportsTablet": true,
      "bundleIdentifier": "com.SEUNOME.tarefas",
      "buildNumber": "1",
      "infoPlist": {
        "ITSAppUsesNonExemptEncryption": false
      }
    },
    "android": {
      "package": "com.SEUNOME.tarefas",
      "versionCode": 1,
      "adaptiveIcon": {
        "backgroundColor": "#E6F4FE",
        "foregroundImage": "./assets/android-icon-foreground.png",
        "backgroundImage": "./assets/android-icon-background.png",
        "monochromeImage": "./assets/android-icon-monochrome.png"
      }
    }
  }
}
```

CAMPO	SIGNIFICADO
`name`	Nome que aparece embaixo do ícone no celular
`slug`	Identificador do projeto no Expo (sem espaços)
`version`	Versão que o **usuário** vê (1.0.0, 1.0.1, 1.1.0...)
`ios.bundleIdentifier` e `android.package`	Identificador **único no mundo** do app, no formato `com.empresa.app`. **Não dá para mudar depois de publicar!**

CAMPO	SIGNIFICADO
`ios.buildNumber` e `android.versionCode`	Número interno da compilação. Cada envio para a loja precisa de um número maior que o anterior
`ITSAppUsesNonExemptEncryption: false`	Responde à pergunta de criptografia da Apple (nosso app só usa criptografia padrão), evitando uma pergunta a cada envio

Troque `SEUNOME` por algo seu, em minúsculas e sem acentos ou espaços (ex.: `com.joaosilva.tarefas`).

Ícone

- Arquivo PNG **quadrado de 1024 x 1024**, sem transparência para o ícone do iOS.
- Ferramentas gratuitas: Canva, Figma, ou peça para uma IA gerar a imagem.
- Substitua `assets/icon.png`. Para o Android, os arquivos `android-icon-*.png` formam o ícone adaptativo (a forma muda conforme o aparelho).

Conferindo se está tudo certo

terminal

```
npx expo-doctor
```

Esse comando faz uma checagem de saúde do projeto. O ideal é aparecer "No issues detected".

🏆 **Desafio 3 (lembre do commit na sua branch):** crie o seu ícone, defina o seu próprio identificador (`com.seunome.tarefas`) e rode o `expo-doctor`. Quem fizer chega ao encontro 4 com o app pronto para ser compilado.

ENCONTRO 4

Build e Google Play

QUI 05/11 · 19:30 ÀS 20:30

NESTE ENCONTRO

1. O que é um "build"?
2. Preparando o EAS
3. Primeiro build: APK para testar
4. Build de produção (AAB)
5. Conta na Google Play Console
6. Criando o app na Play Console (passo a passo)
7. Política de privacidade (obrigatória)

4.1 O que é um "build"?

Até agora o app rodou **dentro do Expo Go**. Para publicar, precisamos de um arquivo próprio, só do nosso app:

SISTEMA	ARQUIVO	USO
Android	**.apk**	Instalar direto no celular para testar
Android	**.aab** (Android App Bundle)	O formato **exigido pela Play Store**
iOS	**.ipa**	Enviado para a App Store / TestFlight

Quem gera esses arquivos é o **EAS Build**, na nuvem do Expo. Você não precisa de Android Studio, nem de Mac para gerar o iOS.

4.2 Preparando o EAS

terminal

```
npm install -g eas-cli
eas login
eas init
```

COMANDO	O QUE FAZ
`npm install -g eas-cli`	Instala a ferramenta de linha de comando do EAS (`-g` = no computador todo)
`eas login`	Entra com sua conta do Expo
`eas init`	Liga este projeto à sua conta no Expo (cria um `projectId` no `app.json`)

O arquivo `eas.json` define os "perfis" de build:

app.json

```
{
  "cli": { "version": "≥ 16.0.0", "appVersionSource": "remote" },
  "build": {
    "preview": {
      "distribution": "internal",
      "android": { "buildType": "apk" }
    },
    "production": {
      "autoIncrement": true
    }
  },
  "submit": { "production": {} }
}
```

- `preview`: gera um **APK** para instalar e testar direto no celular.

- `production`: gera o **AAB** para a loja. O `autoIncrement` aumenta o número da versão interna sozinho a cada build.

4.3 Primeiro build: APK para testar

```terminal
eas build --platform android --profile preview
```

Na primeira vez o EAS vai perguntar se pode **gerar uma keystore** (a "chave" que assina o app). Responda **Yes**. O EAS guarda a chave com segurança para você.

Espere o build na nuvem (de 5 a 20 minutos, dependendo da fila do plano gratuito). Ao terminar, aparece um link e um QR Code. Abra no celular Android, baixe e instale o APK. Agora é um app de verdade, com o seu ícone e nome, fora do Expo Go.

> ⚠️ O plano gratuito do Expo tem limite mensal de builds e fila mais lenta. Por isso: **só gere build quando o app já estiver testado no Expo Go.**

4.4 Build de produção (AAB)

```terminal
eas build --platform android --profile production
```

Quando terminar, baixe o arquivo `.aab` pelo link do terminal ou pelo site expo.dev (seu projeto, aba "Builds").

4.5 Conta na Google Play Console

O QUE	DETALHES
Endereço	play.google.com/console
Custo	**US$ 25, pago uma única vez**
Quem pode	Maiores de 18 anos (veja o aviso abaixo)
Verificação	Documento de identidade e verificação de contato. Pode levar alguns dias, então comece cedo

> ⚠️ **Aviso para menores de 18 anos:** as lojas exigem que o titular da conta de desenvolvedor seja maior de idade. Quem tem menos de 18 anos pode fazer todo o app e a preparação, mas a conta de publicação deve ser de um responsável. Neste workshop, o professor mostra o processo real na conta dele.

Regra importante para contas pessoais novas

Contas pessoais criadas recentemente precisam fazer um **teste fechado com pelo menos 12 testadores, por 14 dias seguidos**, antes de poder pedir a publicação em produção. Contas de empresa (organização) não têm essa exigência. Confira sempre as regras atuais na própria Play Console, pois elas mudam.

4.6 Criando o app na Play Console (passo a passo)

1. **Criar app:** botão "Criar app". Informe nome, idioma padrão (Português do Brasil), tipo "App", gratuito ou pago, e aceite as declarações.
2. **Configuração inicial (painel):** a Play Console mostra uma lista de tarefas obrigatórias:
   - **Política de privacidade:** link público (veja a seção 4.7).
   - **Acesso ao app:** informar se há login (o nosso não tem).
   - **Anúncios:** o app tem anúncios? (o nosso não).
   - **Classificação de conteúdo:** responder um questionário.
   - **Público-alvo:** faixa etária. Atenção: marcar público infantil ativa regras extras.
   - **Segurança dos dados:** informar quais dados o app coleta (o nosso não coleta nada, tudo fica no aparelho).
3. **Ficha da loja ("Presença na loja"):**

ITEM	REGRA
Nome do app	Até 30 caracteres
Descrição curta	Até 80 caracteres
Descrição completa	Até 4.000 caracteres
Ícone	PNG 512 x 512
Imagem de destaque	1024 x 500
Capturas de tela	No mínimo 2, do celular

4. **Testes:** no menu "Testes" existem as trilhas **Interno** (até 100 testadores, disponível em minutos), **Fechado** (testadores por e-mail ou grupo, usado para a regra dos 12 testadores), **Aberto** (qualquer pessoa pode entrar) e depois **Produção**.
5. **Criar versão:** em uma trilha, "Criar nova versão", envie o arquivo `.aab`, escreva as notas da versão e salve.
6. **Enviar para análise:** depois de resolver todas as pendências do painel, envie para revisão. A análise pode levar de algumas horas a alguns dias.

⚠️ **Atenção:** o **primeiro** envio do app precisa ser feito manualmente, subindo o `.aab` pelo site. Depois dele, dá para automatizar com `eas submit --platform android` (que exige uma chave de conta de serviço do Google, assunto avançado).

Como tirar as capturas de tela

Com o app aberto no celular, tire prints normalmente (são aceitos prints do próprio aparelho). Mostre o app **com tarefas cadastradas**, não vazio. Boas telas: lista com itens, item concluído, filtros.

4.7 Política de privacidade (obrigatória)

Toda loja exige uma **URL pública** com a política de privacidade, mesmo para apps que não coletam nada. Modelo simples para o nosso app:

```texto
Política de Privacidade - Conect Tarefas

O aplicativo Conect Tarefas não coleta, não compartilha e não envia
nenhum dado pessoal. As tarefas criadas ficam armazenadas apenas no
dispositivo do usuário e são apagadas ao desinstalar o aplicativo.

Contato: seuemail@exemplo.com
Última atualização: dd/mm/aaaa
```

Onde hospedar de graça: uma página pública do Notion, do Google Sites ou do GitHub Pages. Copie o link e cole nos formulários das lojas.

ENCONTRO 5

Apple App Store

SÁB 07/11 · 13:00 ÀS 14:00

NESTE ENCONTRO

1. Diferenças entre Google e Apple
2. Conta Apple Developer
3. Criando o app no App Store Connect
4. Build do iOS com o EAS
5. Enviando para a Apple e testando no TestFlight
6. Preenchendo a página do app (versão para a App Store)
7. Motivos comuns de rejeição (e como evitar)
8. Atualizando o app depois de publicado
9. Checklist final de publicação
10. Próximos passos

5.1 Diferenças entre Google e Apple

	GOOGLE PLAY	APPLE APP STORE
Custo	US$ 25 uma vez	**US$ 99 por ano**
Revisão	Horas a poucos dias	Normalmente 1 a 2 dias, às vezes mais
Arquivo	.aab	.ipa
Teste antes de publicar	Trilhas interna e fechada	**TestFlight**
Precisa de Mac?	Não	Não, se usar o EAS Build
Rigor da revisão	Médio	Alto

💡 **Nota do workshop:** na aula de 07/11 o professor **não criará** uma conta Apple Developer (ela custa US$ 99/ano). O processo da Apple é apresentado de forma demonstrativa, para você ver que é possível e poder repetir depois seguindo este capítulo e o tutorial do blog do professor: https://blog.otaviorafael.com.br/post/como-publicar-seu-app-react-native-expo-na-play-store-e-app-store

5.2 Conta Apple Developer

1. Tenha um **Apple ID** com autenticação em duas etapas ativada.
2. Acesse **developer.apple.com/programs** e faça a inscrição no **Apple Developer Program** (US$ 99/ano). A aprovação pode demorar de algumas horas a alguns dias.
3. Titular maior de 18 anos (mesma regra do Google).

Sem essa conta paga não é possível gerar um app iOS instalável nem publicar.

5.3 Criando o app no App Store Connect

Acesse **appstoreconnect.apple.com**, vá em "Apps", botão **+ > Novo app**:

CAMPO	O QUE COLOCAR
Plataformas	iOS
Nome	Nome público do app (único na loja)
Idioma principal	Português (Brasil)
ID do pacote (Bundle ID)	O mesmo `ios.bundleIdentifier` do `app.json`
SKU	Código interno livre, ex.: `conecttarefas001`

5.4 Build do iOS com o EAS

```terminal
eas build --platform ios --profile production
```

O EAS pede o login da sua conta Apple Developer e **cria sozinho** os certificados e perfis de provisionamento (as "assinaturas" do app), uma tarefa que antes era um pesadelo. Responda "Yes" às perguntas de geração de credenciais. Se precisar rever as credenciais: `eas credentials`.

5.5 Enviando para a Apple e testando no TestFlight

```terminal
eas submit --platform ios
```

O comando envia o `.ipa` para o App Store Connect. Ele pede o login Apple e o app a ser usado (ou o `ascAppId`, o número do app no App Store Connect).

Depois de processado (alguns minutos), o build aparece na aba **TestFlight**. Lá você convida testadores (equipe interna ou por e-mail/link) que instalam o app pelo aplicativo **TestFlight**. É o equivalente ao teste fechado do Google e é a melhor forma de ver o app no iPhone antes de lançar.

5.6 Preenchendo a página do app (versão para a App Store)

ITEM	DETALHES
Capturas de tela	Tamanho de iPhone de 6,9" (ex.: 1320 x 2868); a Apple pede os tamanhos atuais, confira no site
Texto promocional e descrição	Explique o que o app faz
Palavras-chave	Até 100 caracteres, separadas por vírgula
URL de suporte	Página ou e-mail onde o usuário pede ajuda
URL da política de privacidade	A mesma usada no Google
Privacidade do app	Declarar os dados coletados (no nosso caso: nenhum)
Classificação etária	Questionário rápido
Direitos autorais	Ex.: `2026 Seu Nome`
Notas para a revisão	Explique como usar o app; se não há login, diga isso

Escolha o build enviado, clique em **Adicionar para revisão** e depois **Enviar para revisão**.

5.7 Motivos comuns de rejeição (e como evitar)

MOTIVO	COMO EVITAR
App muito simples ("funcionalidade mínima", regra 4.2 da Apple)	Faça o app útil e polido. Um app de tarefas precisa ser mais do que uma tela de exemplo: cuide do visual e das funções
App que fecha ou trava	Teste bem no TestFlight, em mais de um aparelho
Falta de política de privacidade	Inclua a URL válida
Capturas de tela que não mostram o app de verdade	Use prints reais do app funcionando
Informações incompletas	Preencha todos os campos e as notas da revisão
Identificador ou nome copiando marca de terceiros	Use um nome original

Se for rejeitado, **não se desespere**: a Apple e o Google explicam o motivo. Corrija, gere um novo build e reenvie. Acontece com todo mundo.

5.8 Atualizando o app depois de publicado

Cada atualização enviada às lojas precisa de **versão nova**:

1. Mude `version` no `app.json` (ex.: `1.0.0` para `1.0.1`).
2. Rode o build de produção de novo (o `autoIncrement` cuida do número interno).
3. Envie para as lojas e passe pela revisão novamente.

Bônus: EAS Update. Para mudanças só de código JavaScript (um texto, uma cor, uma correção), é possível enviar a atualização direto para os usuários sem passar pela revisão da loja. Os comandos básicos são `npx expo install expo-updates`, `eas update:configure` e `eas update`. Mudanças que mexam em código nativo, permissões ou bibliotecas novas ainda exigem um novo build e nova revisão.

5.9 Checklist final de publicação

Projeto

- [ ] App testado no Expo Go e em build de teste (APK / TestFlight)
- [ ] Nome, ícone e identificador definidos no `app.json`
- [ ] `npx expo-doctor` sem erros

Contas

- [ ] Conta Expo (gratuita)
- [ ] Google Play Console (US$ 25 uma vez)
- [ ] Apple Developer Program (US$ 99 por ano)

Material da loja

- [ ] Ícone 1024 x 1024 e ícone 512 x 512
- [ ] Imagem de destaque 1024 x 500 (Google)

- [ ] Capturas de tela reais (Google e Apple)
- [ ] Descrição curta e completa
- [ ] URL da política de privacidade
- [ ] URL/e-mail de suporte

Envio

- [ ] Build de produção gerado (`.aab` e `.ipa`)
- [ ] Testes internos/TestFlight feitos
- [ ] Questionários de classificação, dados e privacidade respondidos
- [ ] Enviado para revisão

5.10 Próximos passos

Ideias para evoluir o Conect Tarefas e aprender mais:

- **Navegação entre telas:** biblioteca Expo Router (já usada nos templates mais completos do Expo)
- **Editar tarefa**, **data de entrega**, **categorias** e **notificações**
- **Modo escuro** com `useColorScheme`
- **Login e dados na nuvem** com Firebase ou Supabase
- **Monetização:** anúncios ou compras dentro do app
- Criar o **seu próprio app** (um quiz, um diário, um controle de gastos) e publicar

Leitura complementar do professor: o artigo "Como publicar seu app React Native Expo na Play Store e App Store" repete o passo a passo de publicação em formato de texto, para consultar depois das aulas: https://blog.otaviorafael.com.br/post/como-publicar-seu-app-react-native-expo-na-play-store-e-app-store

Artigo do blog | Portfólio do professor | Documentação do Expo | React Native

Portfólio do professor Otávio Rafael: https://portfolio.otaviorafael.com.br/#about

Onde estudar: documentação do Expo (docs.expo.dev), documentação do React Native (reactnative.dev), canal do Expo no YouTube.

Apêndice A: Comandos do workshop

COMANDO	PARA QUE SERVE
`node -v` / `npm -v`	Ver as versões instaladas
`npx create-expo-app@latest nome --template blank`	Criar um projeto novo
`npx expo start`	Rodar o servidor de desenvolvimento
`npx expo start --tunnel`	Rodar usando túnel (quando o Wi-Fi atrapalha)
`npx expo start --clear`	Rodar limpando o cache
`npx expo install pacote`	Instalar biblioteca compatível com o Expo
`npx expo-doctor`	Checar problemas no projeto
`npm install -g eas-cli`	Instalar o EAS
`eas login` / `eas whoami`	Entrar na conta / ver quem está logado
`eas init`	Ligar o projeto ao Expo
`eas build --platform android --profile preview`	Gerar APK de teste
`eas build --platform android --profile production`	Gerar AAB para a Play Store
`eas build --platform ios --profile production`	Gerar IPA para a App Store
`eas submit --platform ios`	Enviar o build para o App Store Connect
`eas credentials`	Gerenciar certificados e chaves
`eas update`	Enviar atualização só de JavaScript

Apêndice B: Resolvendo problemas

PROBLEMA	SOLUÇÃO
`comando não encontrado: node`	Reinstale o Node.js e reabra o terminal
QR Code não abre	Mesmo Wi-Fi? Tente `npx expo start --tunnel`
Tela vermelha de erro	Leia a primeira linha da mensagem. Costuma indicar o arquivo e a linha do erro
"Text strings must be rendered within a `<Text>`"	Há texto solto fora de um `<Text>`
Alterações não aparecem	Salve o arquivo; balance o celular e escolha "Reload", ou rode `--clear`
Erro ao instalar biblioteca	Use `npx expo install` em vez de `npm install`

PROBLEMA	SOLUÇÃO
Build do EAS falhou	Abra o link do build no expo.dev e leia o log. Costuma ser identificador faltando ou erro no `app.json`
"Package name already exists" no Google	O `android.package` já foi usado por outra pessoa. Escolha outro

Apêndice C: Glossário

- **Componente:** peça reutilizável de interface, escrita como função.
- **JSX:** sintaxe que mistura JavaScript e marcação, usada dentro dos componentes.
- **Estado:** dados que, ao mudar, atualizam a tela.
- **Hook:** função do React que começa com `use` (`useState`, `useEffect`).
- **Build:** processo que transforma seu código em um app instalável.
- **Keystore:** chave digital que assina seu app Android.
- **AAB / APK / IPA:** formatos de arquivo de app (Android loja, Android direto, iOS).
- **TestFlight:** sistema da Apple para testar apps antes de publicar.
- **Bundle ID / Package name:** identificador único do app.
- **Revisão:** análise feita pela loja antes de liberar o app.

Apêndice D: Guia das tags do React Native

Uma **tag** é um componente escrito entre `< >` dentro do JSX, como `<View>` ou `<Text>`. Este guia explica **cada tag usada no Conect Tarefas**: o que ela é, o que faz e onde aparece no `App.js`.

TAG	É COMO...	PARA QUE SERVE	DE ONDE VEM
`View`	`div`	Caixa que agrupa e organiza outros componentes	`react-native`
`Text`	`p`, `span`, `h1`	Mostra qualquer texto	`react-native`
`TextInput`	`input`	Campo para digitar	`react-native`
`TouchableOpacity`	`button`	Qualquer coisa que reage ao toque	`react-native`
`FlatList`	`ul` com `map`	Lista longa que rola	`react-native`
`StatusBar`	(barra do celular)	Controla hora, bateria e sinal	`expo-status-bar`

O topo do `App.js` importa tudo isso:

App.js

```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, TextInput, TouchableOpacity, FlatList } from 'react-native';
```

D.1 `<View>`: a caixa

O que é: o componente mais usado. Ele não mostra nada sozinho: serve para **agrupar e posicionar** outros componentes, como uma `div`.

No projeto: a tela inteira (container), a linha do campo com o botão **+**, a linha dos filtros e o card de cada tarefa.

App.js

```
<View style={styles.linhaNova}>
  <TextInput style={styles.input} />
  <TouchableOpacity style={styles.botao}>
    <Text>+</Text>
  </TouchableOpacity>
</View>
```

- `style`: define tamanho, cor e posição.
- `flexDirection: 'row'` coloca os filhos lado a lado. Sem isso, ficam em coluna.

D.2 `<Text>`: todo texto

O que é: tudo o que aparece escrito na tela precisa estar dentro de um `Text`. A cor, o tamanho e o peso vêm do `style`.

No projeto: o título "Conect Tarefas", o contador de concluídas, o **+** do botão, o nome de cada filtro e o título de cada tarefa.

App.js

```
<Text style={styles.contador}>
  {qtdFeitas} de {tarefas.length} concluídas
</Text>
```

- `style`: `fontSize`, `color`, `fontWeight`.
- Use `{chaves}` para colocar variáveis e contas dentro do texto.

Erro comum: escrever texto solto dentro de uma `View`. O app mostra "Text strings must be rendered within a `<Text>`". A solução é envolver o texto com `<Text>`.

D.3 `<TextInput>`: campo de digitar

O que é: o campo onde a pessoa digita. O que foi digitado fica guardado no estado (`useState`), como vimos no encontro 2.

No projeto: o campo "Nova tarefa...". O texto digitado vai para a variável `texto`, e apertar **Enter** no teclado também adiciona a tarefa.

App.js

```
<TextInput
  style={styles.input}
  placeholder="Nova tarefa..."
  value={texto}
  onChangeText={setTexto}
  onSubmitEditing={adicionar}
/>
```

PROP	O QUE FAZ
`placeholder`	Texto de dica que aparece quando o campo está vazio
`value`	O valor atual do campo (vem do estado)
`onChangeText`	Função chamada a cada letra digitada (atualiza o estado)
`onSubmitEditing`	Função chamada ao apertar Enter no teclado

D.4 `<TouchableOpacity>` : tudo que é tocável

O que é: deixa qualquer coisa tocável. Quando a pessoa toca, o componente fica mais transparente e chama a função do `onPress`. No React Native não existe a tag `button`, então usamos esta.

No projeto: o botão **+** (adicionar), cada filtro (todas, pendentes, feitas), tocar na tarefa para marcar como feita e a lixeira para apagar.

App.js

```
<TouchableOpacity style={styles.botao} onPress={adicionar}>
  <Text style={styles.botaoTexto}>+</Text>
</TouchableOpacity>
```

- `onPress`: o que fazer ao tocar (é o `onClick` da web).
- `style`: aparência do botão.
- O texto do botão vai **dentro**, em um `Text`.

D.5 `<FlatList>` : a lista que rola

O que é: uma lista que rola sozinha e desenha **só os itens visíveis na tela**. Por isso continua rápida mesmo com muitas tarefas. Funciona como um `map`, mas otimizada para o celular.

No projeto: mostra as tarefas já filtradas. Cada item vira um card, e quando não há nenhuma aparece "Nenhuma tarefa aqui".

App.js

```
<FlatList
  data={visiveis}
  keyExtractor={(item) => item.id}
  renderItem={({ item }) => (
    <Text>{item.titulo}</Text>
  )}
/>
```

PROP	O QUE FAZ
`data`	A lista de dados (aqui, as tarefas visíveis)
`keyExtractor`	Devolve o `id` único de cada item
`renderItem`	Como desenhar cada item
`ListEmptyComponent`	O que mostrar quando a lista está vazia

D.6 `<StatusBar>`: a barra do celular

O que é: controla a barra do topo do celular (hora, bateria e sinal). Ela **não desenha nada dentro do seu app**, só ajusta a cor dos ícones do sistema.

No projeto: fica no fim do `App.js`, dentro da `View` principal.

App.js

```
<View style={styles.container}>
  ...
  <StatusBar style="auto" />
</View>
```

- `style="auto"`: escolhe ícones claros ou escuros conforme o fundo.
- Vem do pacote `expo-status-bar`, **não** do `react-native`. Por isso o import é separado.

D.7 Como as tags se encaixam

Uma tela é uma árvore: tags dentro de tags. Esta é a estrutura do Conect Tarefas:

estrutura

```
View                    container da tela
  Text                  título
  Text                  contador
  View                  linha nova
    TextInput           digitar
    TouchableOpacity    botão +
  View                  filtros
    TouchableOpacity    cada filtro
  FlatList              lista
    View                card da tarefa
      TouchableOpacity  marcar como feita
      TouchableOpacity  apagar
  StatusBar             barra do celular
```

As 3 regras de ouro:

1. Tudo começa em uma `View`: ela é a raiz e guarda o resto.
2. Texto só dentro de `Text`: nunca solto dentro de uma `View`.
3. Toque só com `TouchableOpacity`: ele dispara o `onPress`.

D.8 Também aparecem, mas não são tags

Estas peças estão no `App.js`, mas não desenham nada na tela. São ferramentas de apoio:

NOME	DE ONDE VEM	PARA QUE SERVE
`StyleSheet.create`	`react-native`	Cria o objeto de estilos usado em `style={styles.x}`
`useState`	`react`	Guarda valores que mudam: texto, tarefas, filtro
`useEffect`	`react`	Roda código ao abrir o app e quando a lista muda
`AsyncStorage`	`@react-native-async-storage/async-storage`	Salva as tarefas no celular (encontro 3)

Como diferenciar: tag começa com letra maiúscula e se escreve com `< >`. Hooks (`useState`, `useEffect`) começam com `use` e são chamados como funções.